GROUP PROJECT

TEAM MEMBERS



Aditya Kumar Sharma
(Team leader)
ID:20261651006



Sonam Raj
ID:20261651086



Aditi Choudhary
ID:20261651003

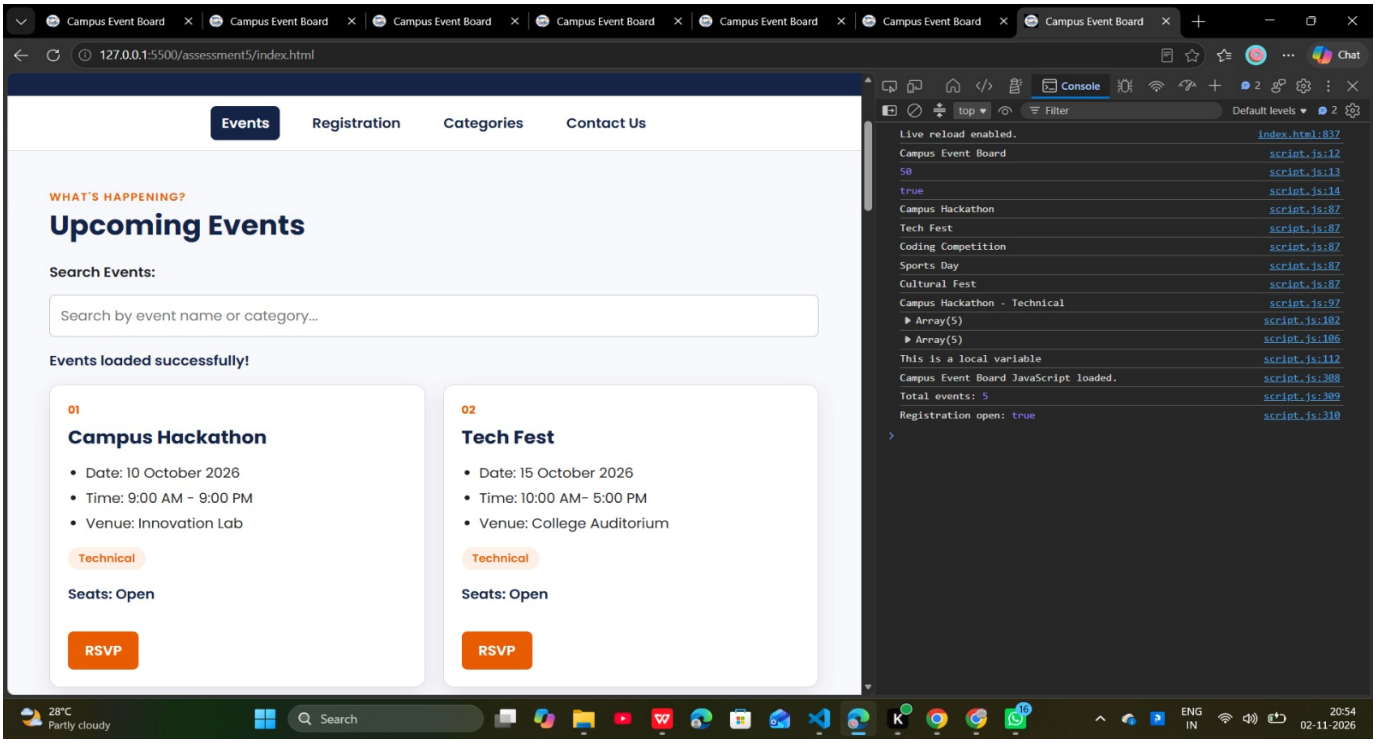


Sujit Kumar
ID:20261651089

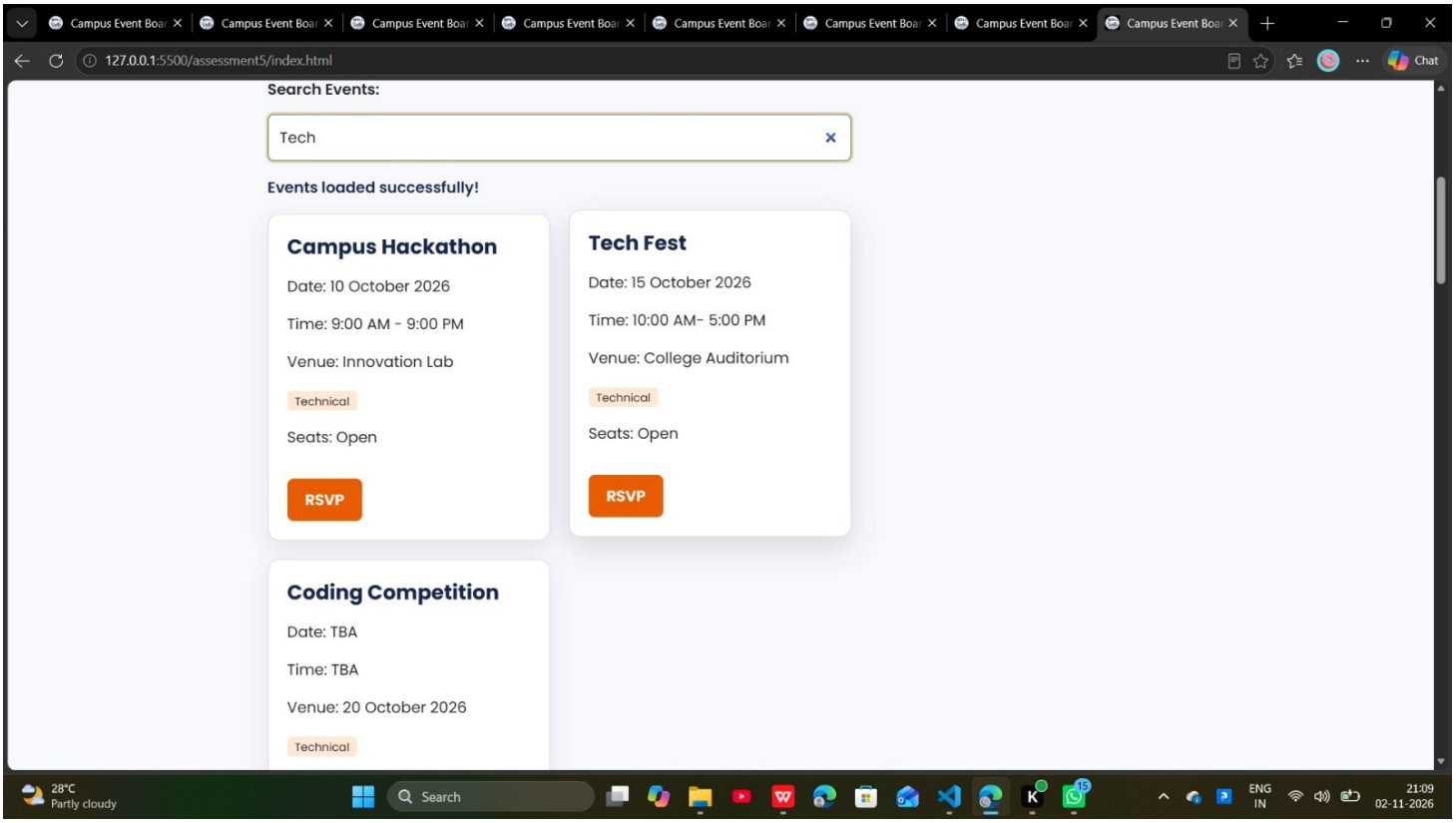


Guru Prashad Panda
ID:20261651039

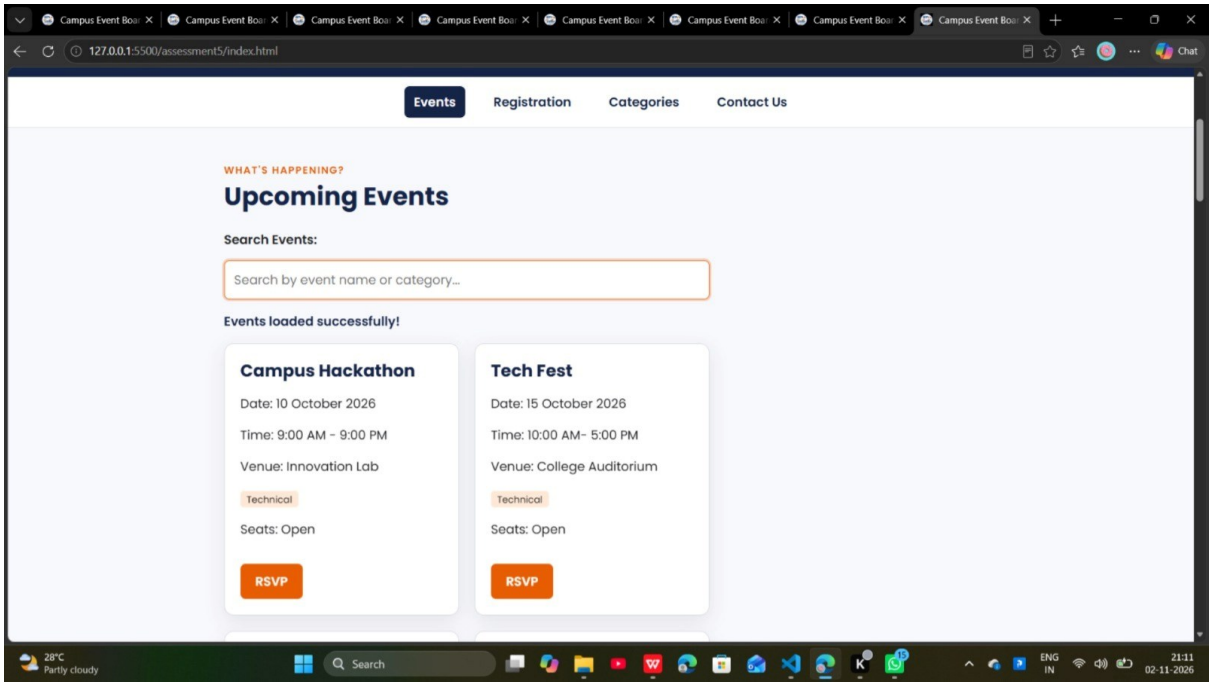
1) Events Rendered from data



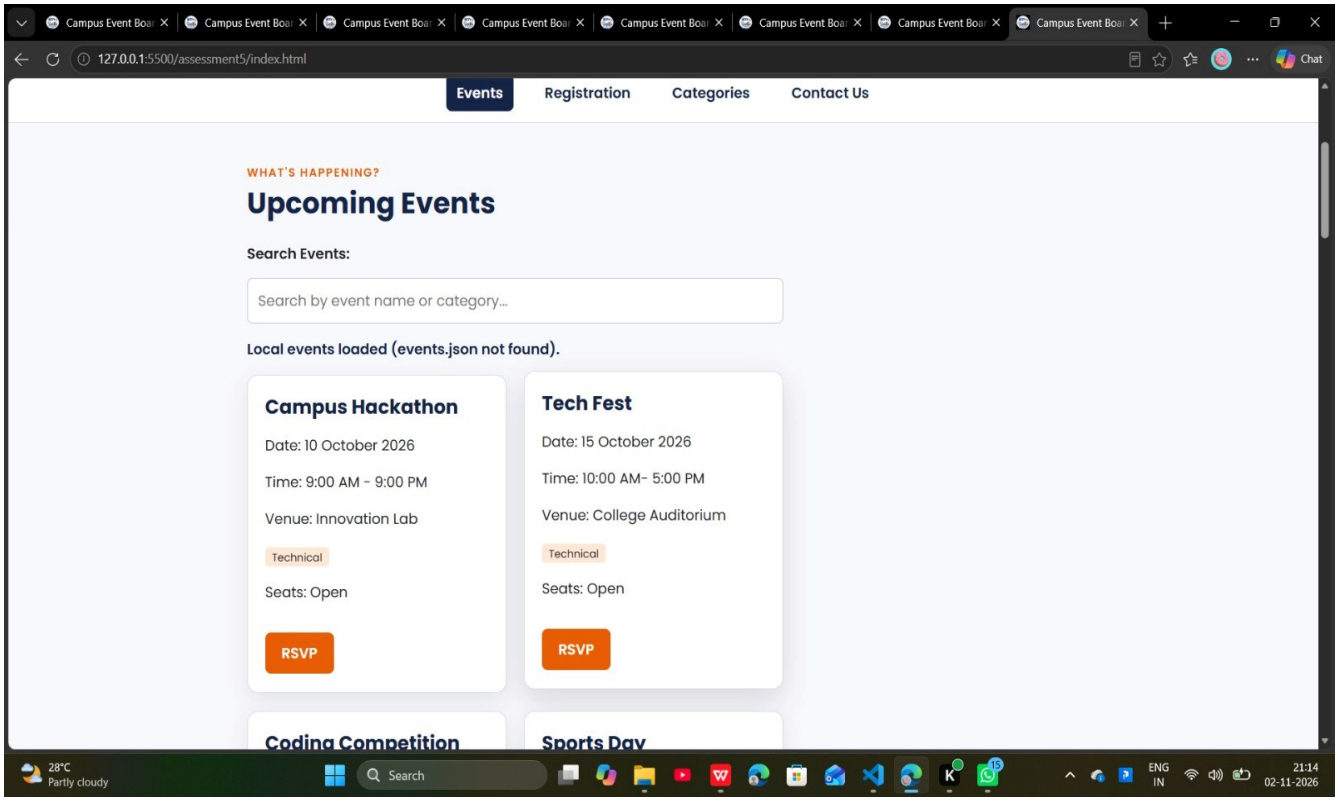
2) Search filter working



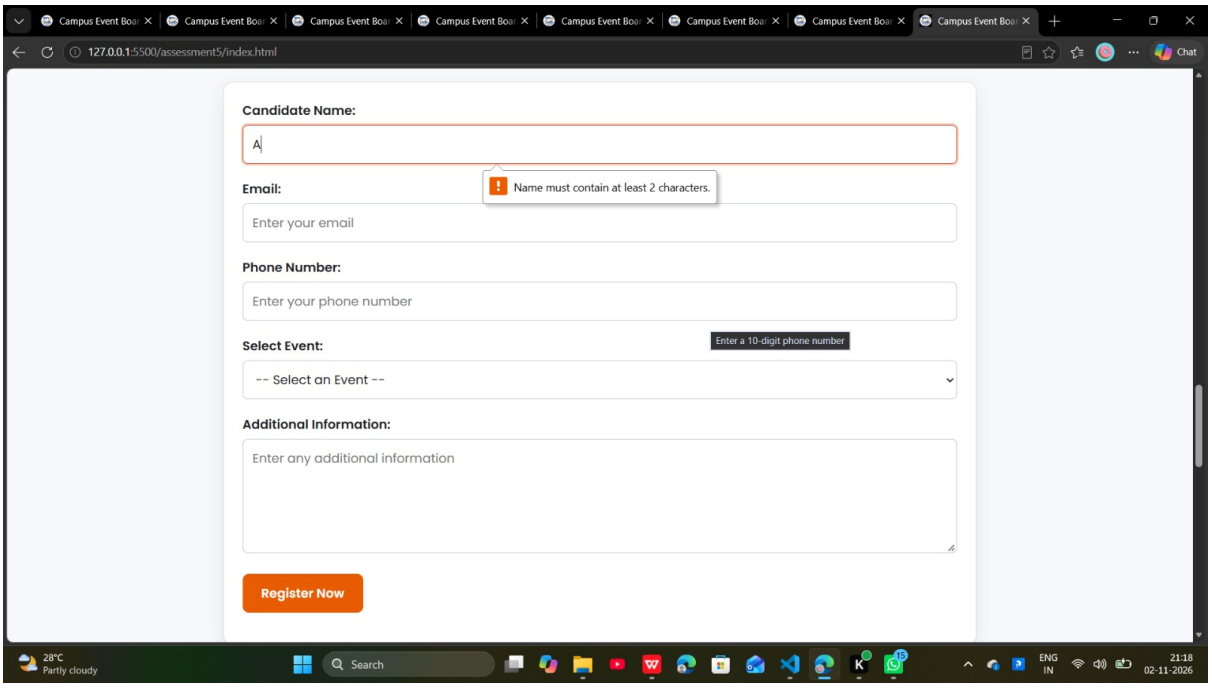
3) Fetch success state - events.json loaded

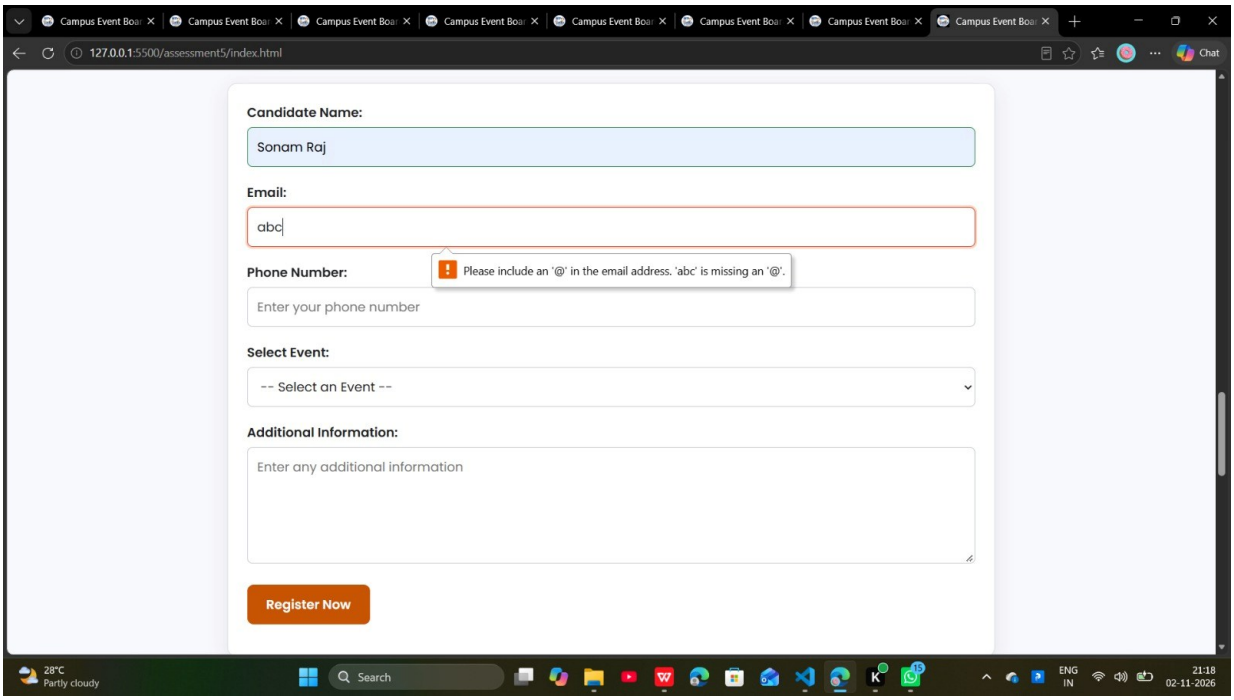


4) Fetch Error State - fallback to local data

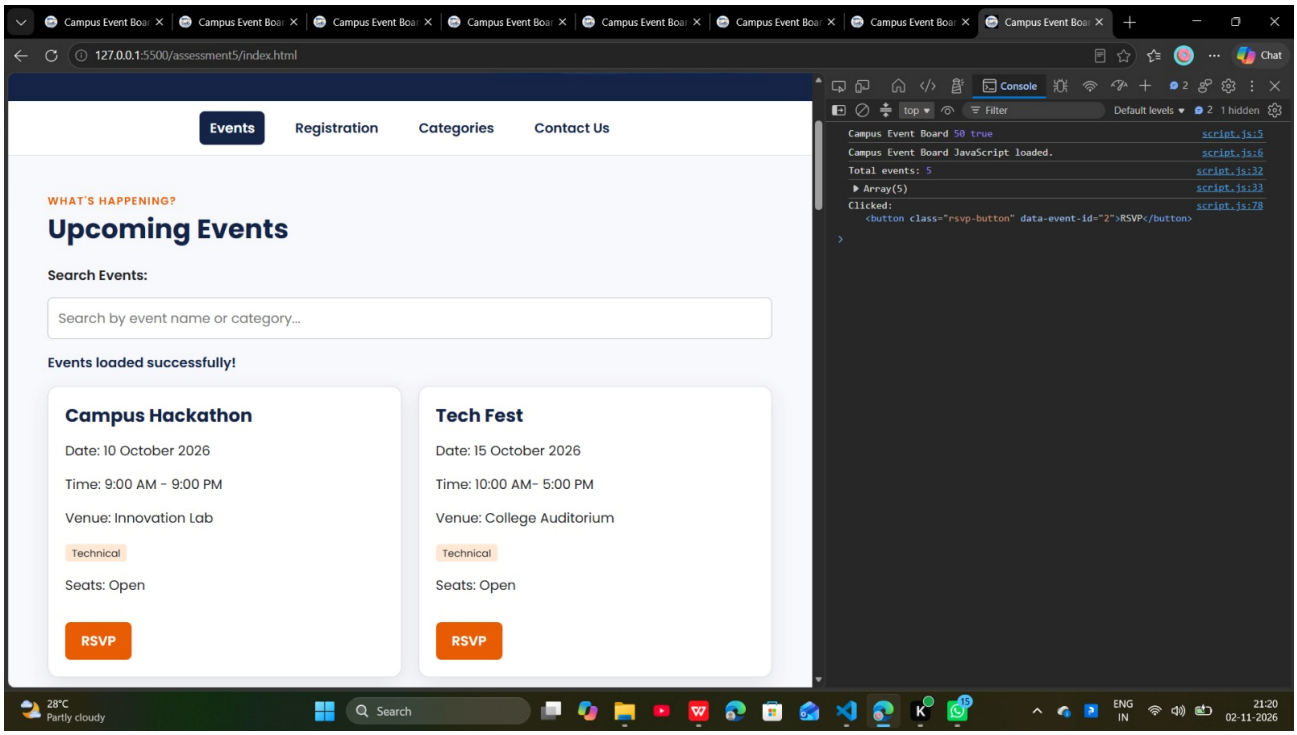


5) Form Validation Error

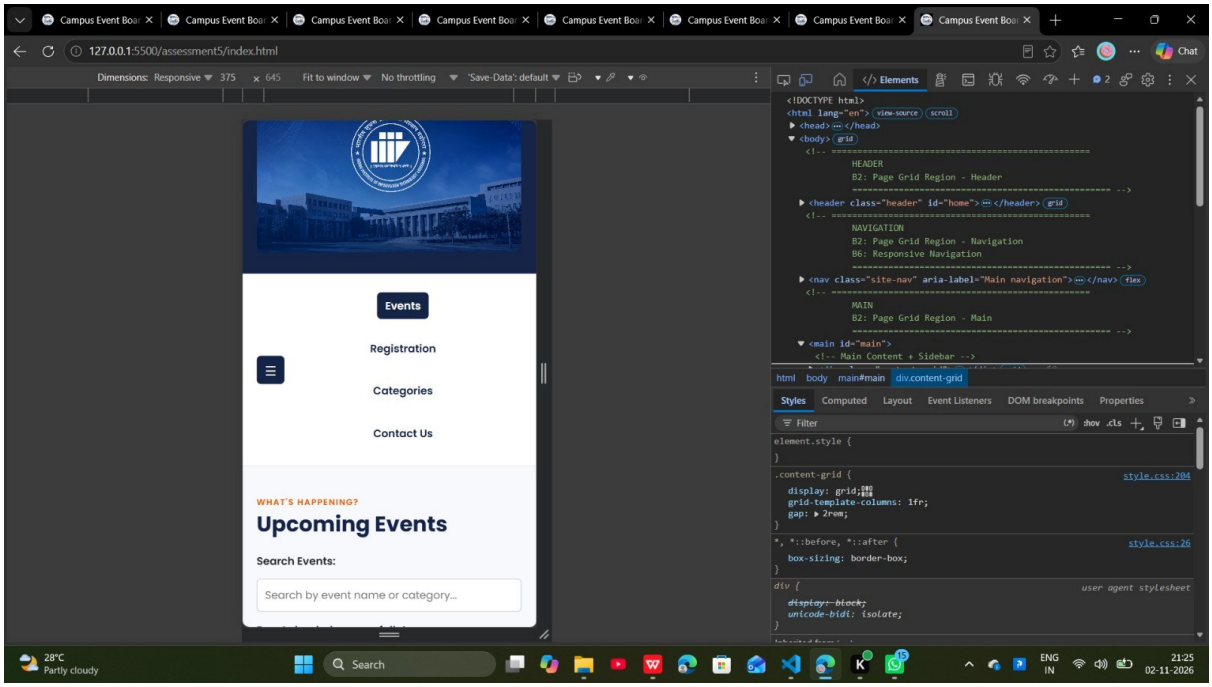




6) DevTools Console Output



7) Mobile Responsive View and fetch success



8) The GitHub repository link

https://github.com/Aadi0622/event-board.git

9) GitHub Pages (Live):

https://aadi0622.github.io/event-board/

Questions & Answers

Answers 3, 4, 5, 7, 8, 9 and 10 are written from the Campus Event Board project files (script.js, index.html, events.json). Answers 1, 2 and 6 are general.

1. `let` vs `const` — when do you use each, and why avoid `var`?

Use `const` by default for any binding that is never reassigned (arrays, objects, functions, DOM references). Note that `const` stops reassignment, not mutation, so you can still `push()` into a const array. Use `let` only when the value must change, such as counters, loop variables or accumulators.

Avoid `var` because it is function-scoped (not block-scoped), so it leaks out of `if` and `for` blocks; it is hoisted and initialised as `undefined`, which hides bugs; and it allows silent re-declaration. `let` and `const` are block-scoped and sit in the temporal dead zone until declared, so mistakes fail loudly.

```
const MAX_SEATS = 100;      // never reassigned
let registered = 0;         // changes over time
registered += 1;
```

2. List the data types you used and give one value of each (string, number, boolean, ...).

```
string     'Web Dev Meetup'
number     42
boolean    true
null       null            (intentionally empty, e.g. no selected event)
undefined  let x;          (declared but not assigned)
object     { id: 1, title: 'Meetup' }
array      [ 'JS', 'CSS', 'HTML' ]   (typeof gives 'object')
function   () => 'hi'
```

3. Show one event object from your array. Why is an array of objects a good model for this page?

From the `events` array in script.js (A5):

```
{
  id: 1,
  title: "Campus Hackathon",
  date: "10 October 2026",
  time: "9:00 AM - 9:00 PM",
  venue: "Innovation Lab",
  category: "Technical",
  seats: 50,
  online: true
}
```

Every event is one record with the same named fields (string, number and boolean values), and the array keeps the records in order and gives built-in methods. This page uses that directly: `renderEvents` loops over the array with `for...of` and builds one card per object; the search box uses `filter` on title, category and venue; the RSVP handler uses `find` to locate an event by `id`; and `map` / `filter` build `eventNames` and `availableEvents`. The shape also matches `events.json`, so fetched data can simply replace the local array

(`events = data`). Adding a new event means adding one object, with no HTML changes.

4. **Paste your control-flow rule from A4. What labels did it produce for your events?**

```
// A4: CONTROL FLOW
function getSeatStatus(event) {
  if (event.seats === 0) return "Full";
  else if (event.seats < 5) return "Few left";
  else return "Open";
}
```

It produces three labels: **Full** (seats is 0), **Few left** (fewer than 5 seats) and **Open** (everything else). `renderEvents` shows the result on each card as "Seats: <label>". With the current data (50, 100, 30, 80 and 120 seats) all five events get the label **Open**. The conditions are checked top to bottom and the first match returns, so 0 seats is caught as Full before the "< 5" test runs.

5. **Show one function (A6) and one arrow function with an array method (A7). What does each return?**

A6 – function:

```
// A6: FUNCTION
function formatEvent(event) {
  return event.title + " - " + event.category;
}
```

It returns a **string**, for example `formatEvent(events[0])` returns `"Campus Hackathon - Technical"`.

A7 – arrow functions with array methods:

```
// A7: ARROW + FILTER/MAP
const availableEvents = events.filter((e) => e.seats > 0); // arrow function + filter()
const eventNames = events.map((e) => e.title); // arrow function + map()
```

`events.filter((e) => e.seats > 0)` returns a **new array of event objects** that still have seats (currently all 5). `events.map((e) => e.title)` returns a **new array of strings**: `["Campus Hackathon", "Tech Fest", "Coding Competition", "Sports Day", "Cultural Fest"]`. The original `events` array is not changed by either.

6. **Which elements did you select, and with which methods? Why prefer `querySelector`?**

```
const list   = document.querySelector('#event-list');
const form   = document.querySelector('#signup-form');
const emailI = document.querySelector('#email');
const status = document.querySelector('.status');
const cards  = document.querySelectorAll('.card');   // NodeList
```

`querySelector` / `querySelectorAll` accept any CSS selector (ids, classes, attributes, descendants), so one consistent API replaces `getElementById`, `getElementsByClassName` and `getElementsByTagName`. It returns `null` if nothing matches, and `querySelectorAll` returns a static NodeList that supports `forEach`, unlike

the live HTMLCollection from the older methods.

7. **Explain event bubbling in one or two lines, using the `event.target` from B5.**

```
// B5: EVENT DELEGATION - USE THE EVENT OBJECT
if (eventContainer) {
  eventContainer.addEventListener("click", function(event) {
    console.log("Clicked:", event.target);
    if (event.target.classList.contains("rsvp-button")) {
      const id = event.target.getAttribute("data-event-id");
      const found = events.find((e) => e.id == id);
      if(found){
        alert("You RSVP'd for: " + found.title);
      }
    }
  });
}
```

A click on an RSVP button fires on the button first and then *bubbles up* through its parent card to `#event-grid`, where the one listener runs. `event.target` is the exact element that was clicked (logged with `console.log("Clicked:", event.target)`); the code checks that it has the `rsvp-button` class, reads its `data-event-id`, and finds the event with `events.find`. Because the listener sits on the parent, it also works for cards that are created later by `renderEvents`.

8. **Describe the three states your `fetch` handles, and how the user sees each.**

```
// B7: FETCH 3 states - Loading, Success, Error
async function loadEvents() {
  try {
    if (fetchStatus) fetchStatus.textContent = "Loading events...";
    const res = await fetch("events.json");
    if (!res.ok) throw new Error("Unable to load events.");
    const data = await res.json();
    events = data;
    renderEvents(events);
    if (fetchStatus) fetchStatus.textContent = "Events loaded successfully!";
  } catch (err) {
    console.error(err);
    renderEvents(events);
    if (fetchStatus) fetchStatus.textContent = "Local events loaded (events.json not found).";
  }
}
loadEvents();
```

1. Loading: before the request, `#fetch-status` is set to "Loading events...", so the user sees a loading message above the (still empty) event grid.

2. Success: the response is OK, `res.json()` is stored in `events`, `renderEvents(events)` draws the cards, and the status line changes to "Events loaded successfully!"

3. Error: if `res.ok` is false or the request fails, the `catch` block logs the error, still renders the local events array as a fallback, and the status line shows "Local events loaded (events.json not found)." So the user still sees cards instead of a blank page. The status paragraph has `aria-live="polite"`, so screen readers announce each change.

9. Show your built-in validation attributes and your custom `setCustomValidity` rule. Why must the server validate again?

Built-in attributes (index.html, registration form):

```
<input type="text"  id="candidate-name" name="candidate-name" minlength="2" required>
<input type="email" id="email" name="email" required>
<input type="tel"   id="phone" name="phone" pattern="[0-9]{10}"
       title="Enter a 10-digit phone number" required>
<select id="event" name="event" required> ... </select>
```

Custom rule (script.js, B8) – the name must have at least 2 characters after trimming spaces, otherwise a custom message is set; an empty string clears it so the field becomes valid again:

```
if (candidateName) {
  candidateName.addEventListener("input", function() {
    if (candidateName.value.trim().length < 2) {
      candidateName.setCustomValidity("Name must contain at least 2 characters.");
    } else {
      candidateName.setCustomValidity("");
    }
  });
}

if (registrationForm) {
  registrationForm.addEventListener("submit", function(e) {
    e.preventDefault();
    if (registrationForm.checkValidity()) {
      if(formMessage) formMessage.textContent = "Registration submitted!";
    } else {
      registrationForm.reportValidity();
    }
  });
```

Why the server must validate again: browser-side validation is only a convenience for the user. It can be bypassed by editing the HTML in dev tools, turning off JavaScript, or sending a request directly (curl, Postman). In this project the `submit` handler only shows "Registration submitted!" and sends nothing to a server, so a real back end must re-check every field (name length, email format, 10-digit phone, valid event) before trusting or storing the data.

10. Give the GitHub repo link (and Pages link if live); confirm a README and readable commit history; paste your final `script.js`.

Repo: https://github.com/<your-username>/<your-repo> (replace with your link)
Pages: https://<your-username>.github.io/<your-repo>/ (replace if live)

README: README.md in the repo root describing the Campus Event Board (IIIT Vadodara), the team, features (event cards, search, RSVP, fetch with fallback, form validation), the file list and how to run it – confirm it is committed before submitting.
Commit history: small commits with clear messages, e.g. "Add events array", "Render event cards", "Add search filter", "Add fetch states", "Add form validation" – confirm with `git log --oneline`.

Final script.js (long lines wrapped to fit the page):

```
// A3: VARIABLES
```

```
const boardTitle = "Campus Event Board";
let seatsLeft = 50;
const isRegistrationOpen = true;
console.log(boardTitle, seatsLeft, isRegistrationOpen);
console.log("Campus Event Board JavaScript loaded.");

// A5: ARRAY OF OBJECTS - Default local data
let events = [
  { id: 1, title: "Campus Hackathon", date: "10 October 2026", time: "9:00 AM - 9:00 PM", venue:
      "Innovation Lab", category: "Technical", seats: 50, online: true },
  { id: 2, title: "Tech Fest", date: "15 October 2026", time: "10:00 AM- 5:00 PM", venue: "College
       Auditorium", category: "Technical", seats: 100, online: true },
  { id: 3, title: "Coding Competition", date: "20 October 2026", time: "11:00 AM- 4:00 PM", venue:
       "Computer Lab", category: "Technical", seats: 30, online: false },
  { id: 4, title: "Sports Day", date: "25 October 2026", time: "8:00 AM- 4:00 PM", venue: "College
       Ground", category: "Sports", seats: 80, online: false },
  { id: 5, title: "Cultural Fest", date: "30 October 2026", time: "5:00 PM- 9:00 PM", venue:
      "Auditorium", category: "Cultural", seats: 120, online: false }
];

// A4: CONTROL FLOW
function getSeatStatus(event) {
  if (event.seats === 0) return "Full";
  else if (event.seats < 5) return "Few left";
  else return "Open";
}

// A6: FUNCTION
function formatEvent(event) {
  return event.title + " - " + event.category;
}

// A7: ARROW + FILTER/MAP
const availableEvents = events.filter((e) => e.seats > 0); // arrow function + filter()
const eventNames = events.map((e) => e.title); // arrow function + map()
console.log("Total events:", events.length);
console.log(eventNames);

// DOM ELEMENTS
const eventContainer = document.querySelector("#event-grid");
const searchInput = document.querySelector("#event-search");
const eventCount = document.querySelector("#event-count");
const navToggle = document.querySelector(".nav-toggle");
const navLinks = document.querySelector(".nav-links");
const fetchStatus = document.querySelector("#fetch-status");

// B3: RENDER + RSVP BUTTON - Read and change content - FIXED for undefined
function renderEvents(list) {
  eventContainer.innerHTML = "";

  if (eventCount) {
    eventCount.textContent = `${list.length} event(s) found.`;
  }

  for (const ev of list) {
    const status = getSeatStatus(ev);
    // Fix for undefined issue - supports both lower and upper case keys
    const dateText = ev.date || ev.Date || "TBA";
    const timeText = ev.time || ev.Time || "TBA";
    const venueText = ev.venue || ev.Venue || ev.location || "TBA";
    const titleText = ev.title || ev.name || "Untitled";
    const categoryText = ev.category || "General";

    const card = document.createElement("article");
    card.className = "event-card";
    card.innerHTML = `
      <h3>${titleText}</h3>
      <p>Date: ${dateText}</p>
```

```
      <p>Time: ${timeText}</p>
      <p>Venue: ${venueText}</p>
      <p><span style="background:#ffe9d6;padding:2px 8px;border-radius:4px;font-
          size:12px;">${categoryText}</span></p>
      <p>Seats: ${status}</p>
      <button class="rsvp-button" data-event-id="${ev.id}">RSVP</button>
    `;
    eventContainer.appendChild(card);
  }
}

// B5: EVENT DELEGATION - USE THE EVENT OBJECT
if (eventContainer) {
  eventContainer.addEventListener("click", function(event) {
    console.log("Clicked:", event.target);
    if (event.target.classList.contains("rsvp-button")) {
      const id = event.target.getAttribute("data-event-id");
      const found = events.find((e) => e.id == id);
      if(found){
        alert("You RSVP'd for: " + found.title);
      }
    }
  });
}

// B4: SEARCH - input event - FIXED: Now filters by title + category + venue so Tech shows 3
    results
if (searchInput) {
  searchInput.addEventListener("input", function(event) {
    const text = event.target.value.toLowerCase().trim();
    if (text === "") {
      renderEvents(events);
      return;
    }
    const filtered = events.filter((e) => {
      const title = (e.title || e.name || "").toLowerCase();
      const category = (e.category || "").toLowerCase();
      const venue = (e.venue || e.Venue || "").toLowerCase();
      return title.includes(text) || category.includes(text) || venue.includes(text);
    });
    renderEvents(filtered);
  });
}

// B6: NAV TOGGLE
if (navToggle && navLinks) {
  navToggle.addEventListener("click", function() {
    navLinks.classList.toggle("is-open");
  });
}

// B7: FETCH 3 states - Loading, Success, Error
async function loadEvents() {
  try {
    if (fetchStatus) fetchStatus.textContent = "Loading events...";
    const res = await fetch("events.json");
    if (!res.ok) throw new Error("Unable to load events.");
    const data = await res.json();
    events = data;
    renderEvents(events);
    if (fetchStatus) fetchStatus.textContent = "Events loaded successfully!";
  } catch (err) {
    console.error(err);
    renderEvents(events);
    if (fetchStatus) fetchStatus.textContent = "Local events loaded (events.json not found).";
  }
}
loadEvents();
```

```
// B8: FORM VALIDATION - setCustomValidity()
const candidateName = document.querySelector("#candidate-name");
const registrationForm = document.querySelector("#registration-form");
const formMessage = document.querySelector("#form-message");

if (candidateName) {
  candidateName.addEventListener("input", function() {
    if (candidateName.value.trim().length < 2) {
      candidateName.setCustomValidity("Name must contain at least 2 characters.");
    } else {
      candidateName.setCustomValidity("");
    }
  });
}

if (registrationForm) {
  registrationForm.addEventListener("submit", function(e) {
    e.preventDefault();
    if (registrationForm.checkValidity()) {
      if(formMessage) formMessage.textContent = "Registration submitted!";
    } else {
      registrationForm.reportValidity();
    }
  });
}
```